

Deep Learning

Lecture 6: Energy-Based Models

Amir Atapour-Abarghouei

amir.atapour-abarghouei@durham.ac.uk

Durham University





① Modelling Trilemma and Manifolds

② Energy-based Models

- Definitions
- GANs and Clustering
- Softmax and Softmin
- Exact Likelihood
- Boltzmann Machines (BM)
- Restricted and Deep BM

③ Probabilistic Diffusion Models

- Approach and Implementation
- Applications

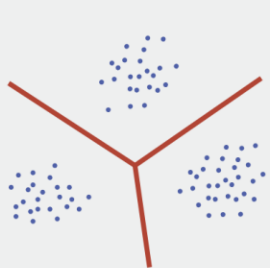
Recap: Generative models



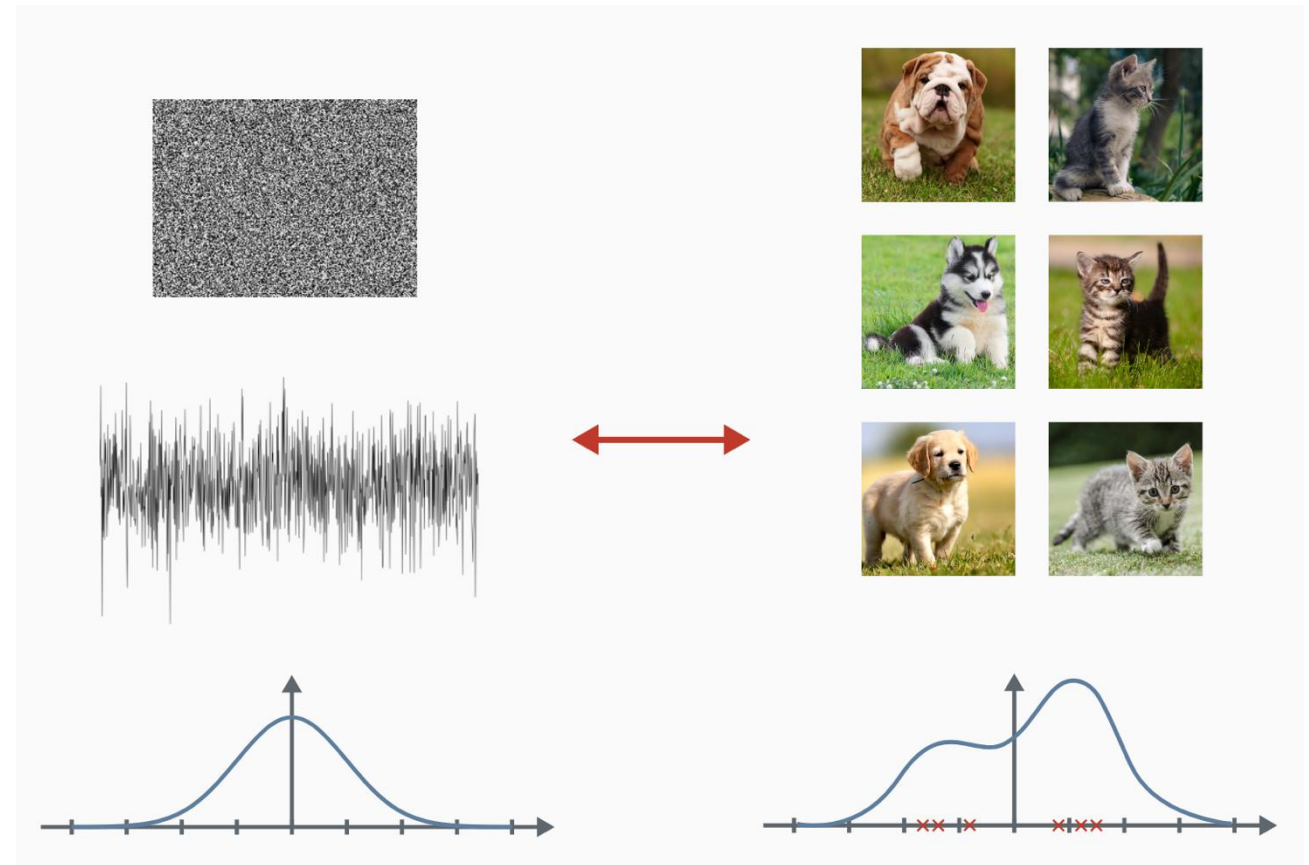
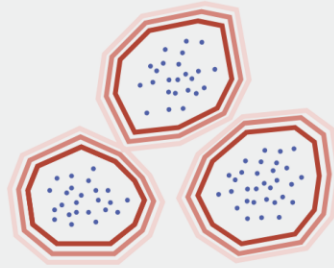
Generative models

- Learn a distribution over a dataset.
- Map between noise and data.
- Noise (prior) can be different dimension to data.

Discriminative modelling
(classification, regression)



Generative modelling
(sampling, density estimation)



Recap: GANs

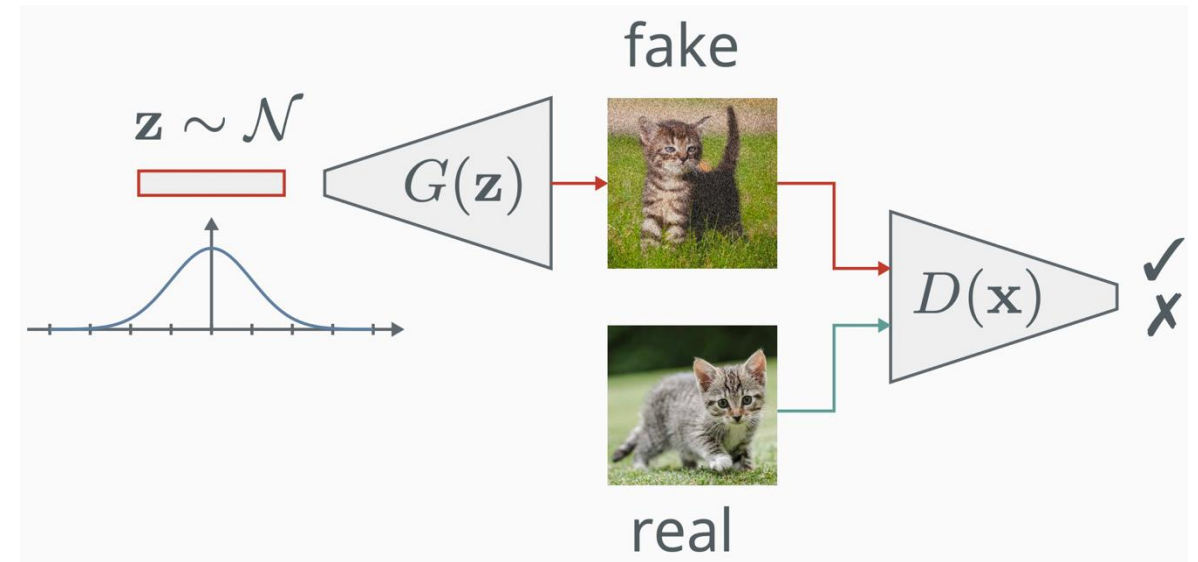


Definition:

A generative adversarial network (GAN) is a non-cooperative zero-sum game where two networks compete against each other

[Goodfellow et al., 2014].

One network $G(z)$ generates new samples, whereas D estimates the probability the sample was from the training data rather than G :



$$\min_{\theta_g} \max_{\theta_d} [\mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))]$$

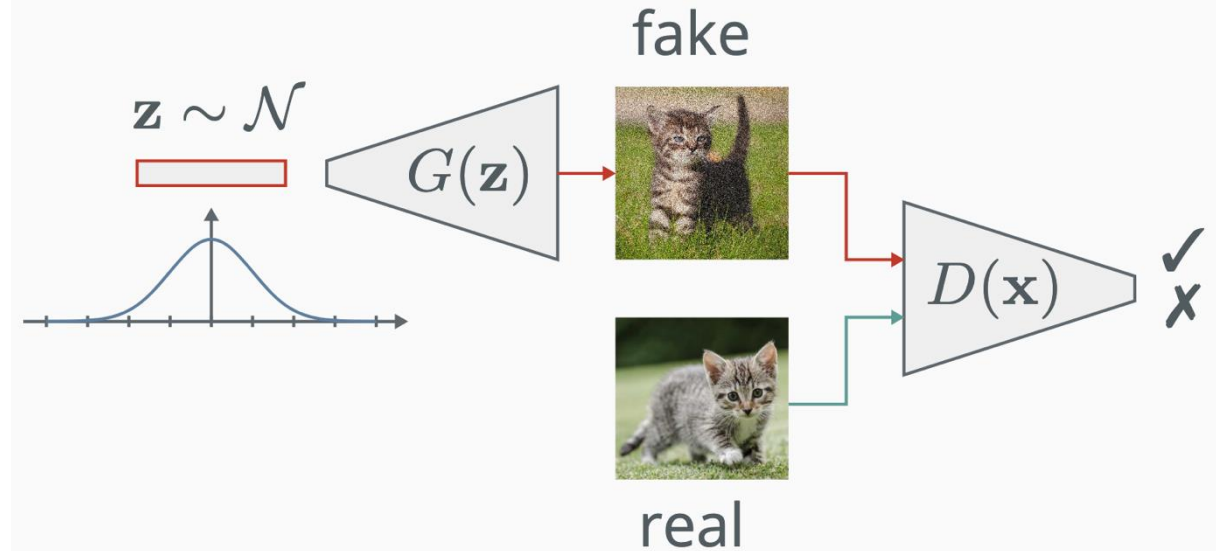
Recap: GANs



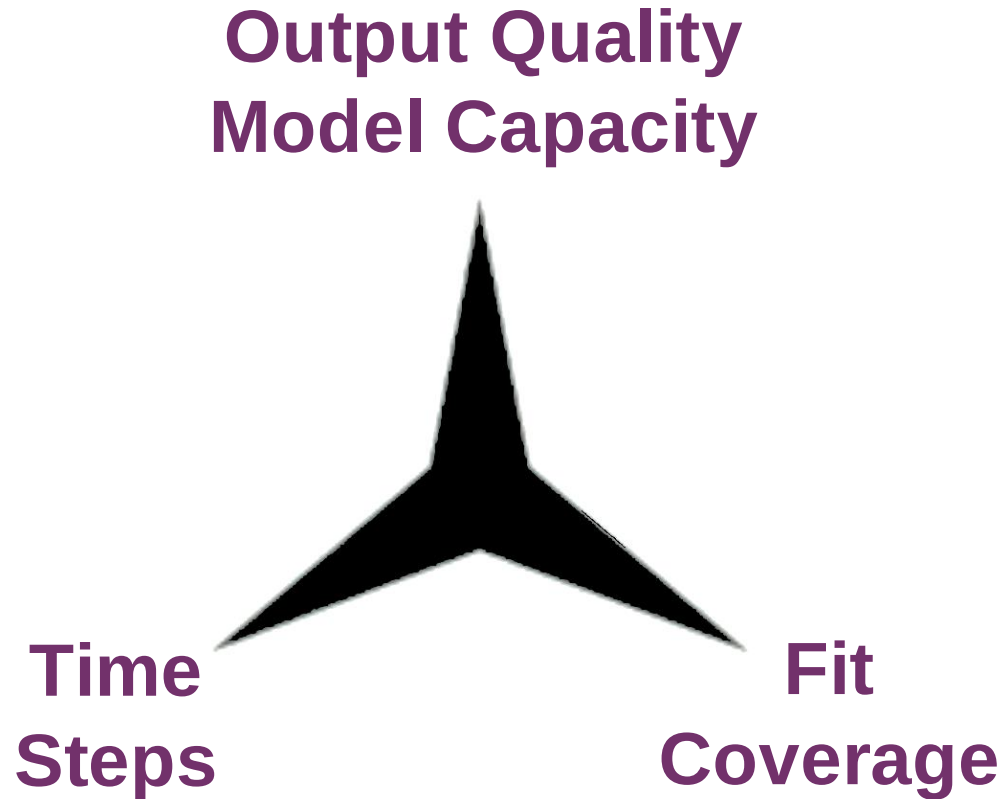
GAN properties

GANs benefit from differentiable data augmentation, but are otherwise have a variety of issues:

- Non-convergence
- Diminishing gradient
- Difficult to balance
- Mode collapse



Generative Modelling Trilemma



GANs benefit from differentiable data augmentation, but are otherwise have a variety of issues:

- Non-convergence
- Diminishing gradient
- Difficult to balance
- Mode collapse

GANs : Trilemma?

- Fast – quick to sample
- Excellent quality
- Poor coverage of data distribution

Autoregressive Generative Models

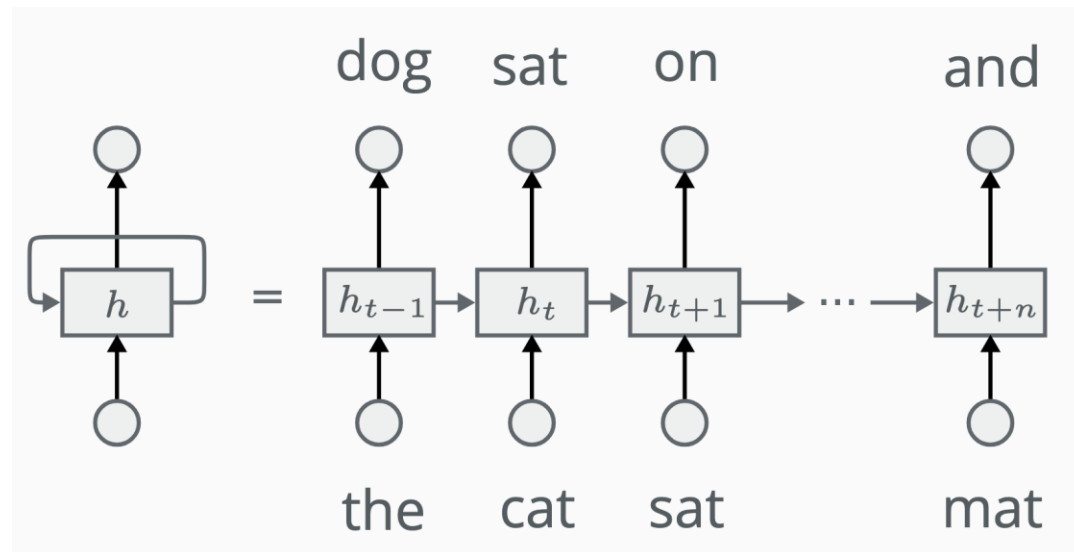


Definition: autoregressive (AR) generative models (e.g. chatGPT)

AR models maximise the likelihood of the training data (excellent mode coverage):

$$p_{\theta}(\mathbf{x}) = p_{\theta}(x_1, \dots, x_N) = \prod_{i=1}^N p_{\theta}(x_i | x_1, \dots, x_{i-1})$$

This is slow due to the sequential nature defined by the chain rule of probability.



Today: Diffusion Probabilistic Models



Definition: diffusion probabilistic models (DPMs)

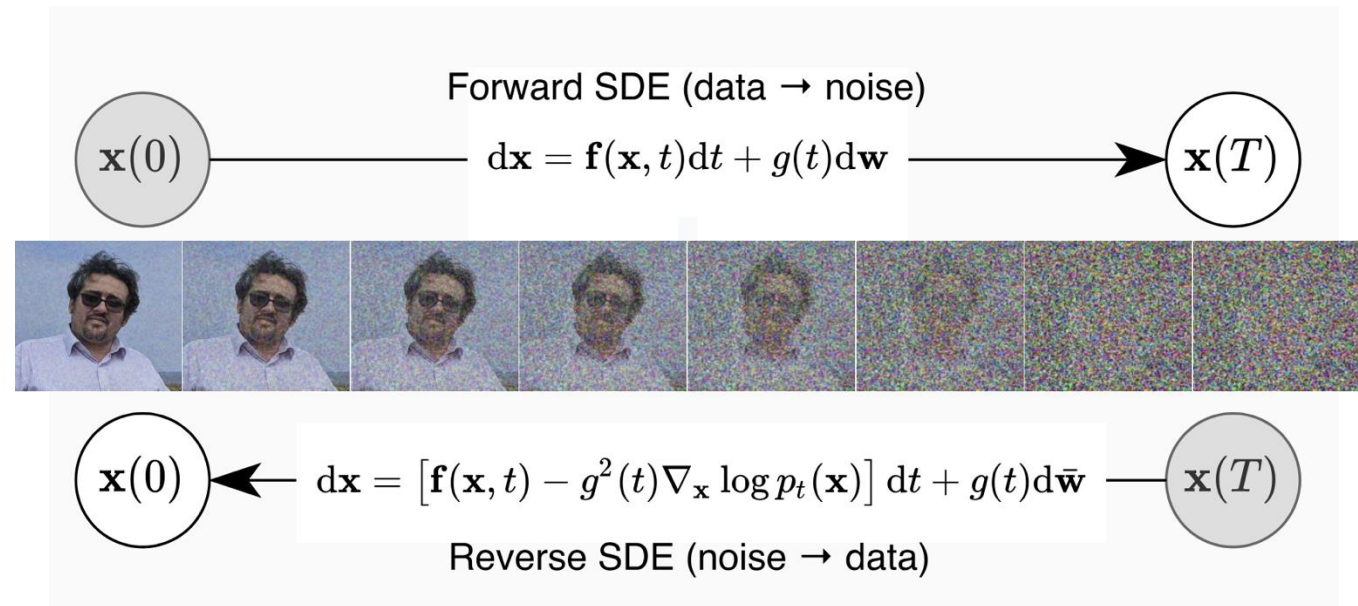
Instead of modelling a sequence of words, model a diffusion of noise in the data space. Define a forward (diffusion) equation:

$$p_{\theta}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t),$$

which can be reversed to sample the model:

$$q(\mathbf{x}_{1:T} | \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1}).$$

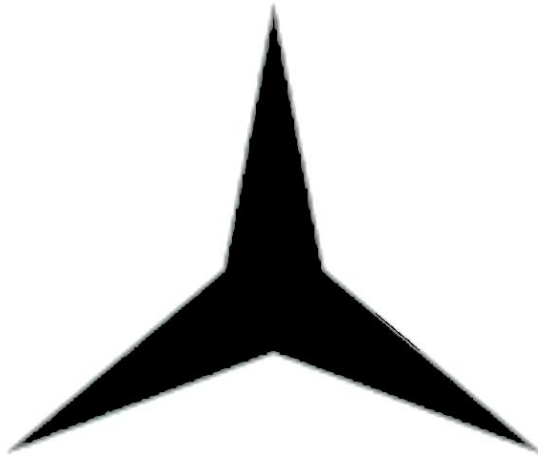
This also requires a long iterative **transformation process**.



Today: Diffusion Probabilistic Models



**Output Quality
Model Capacity**



**Time
Steps**

**Fit
Coverage**

Question: where are these in the trilemma?

- They are slow (lots of iterations)
- They have excellent coverage
- They have excellent quality

...I'd do anything for 3/3...

Definition: manifold

A manifold is a topological space that locally resembles Euclidean space

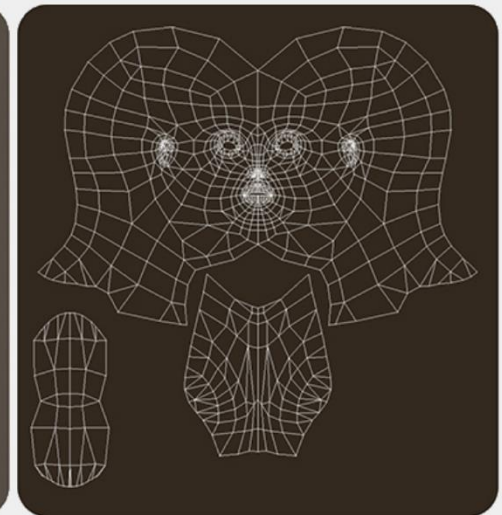
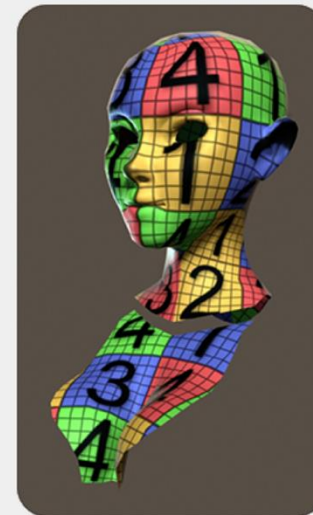
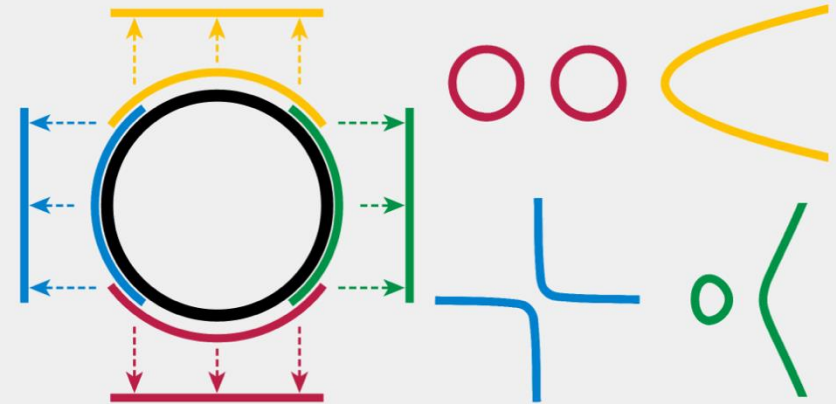
- topological manifold
- differentiable manifold
- Riemannian manifold

Definition: embedding

An embedding is a function ϕ that maps a manifold $\mathcal{M}$ to a new manifold $\mathcal{N}$ in an injective way that preserves its structure:

$$\phi : \mathcal{M} \rightarrow \mathcal{N}$$

Example: manifolds





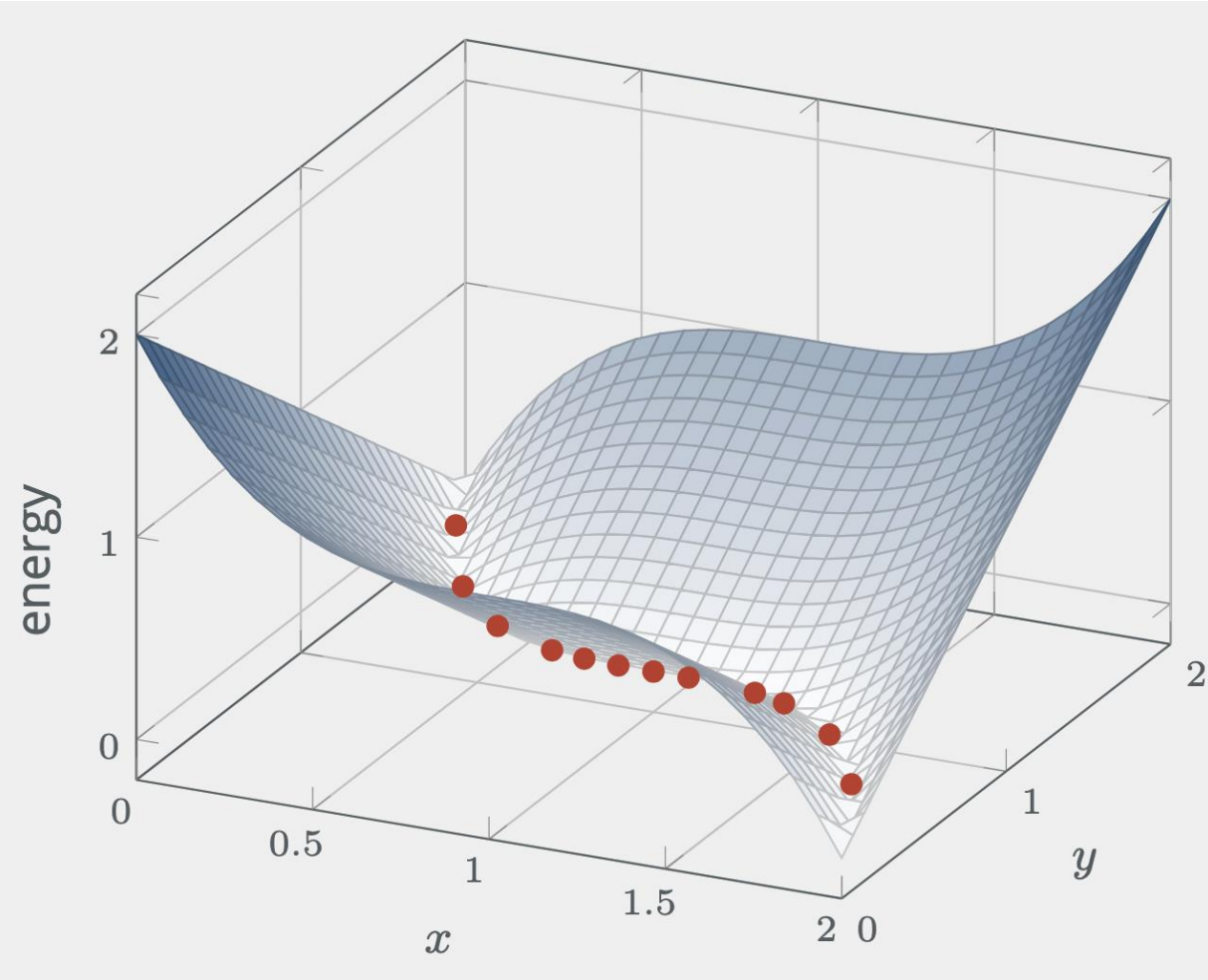
Definition: energy-based models

These are just any function that is happy when you input something that looks like data, and is not happy when you input something that doesn't look like data.

$$E(\mathbf{x}) = 0 \quad \checkmark$$

$$E(\tilde{\mathbf{x}}) > 0 \quad \times$$

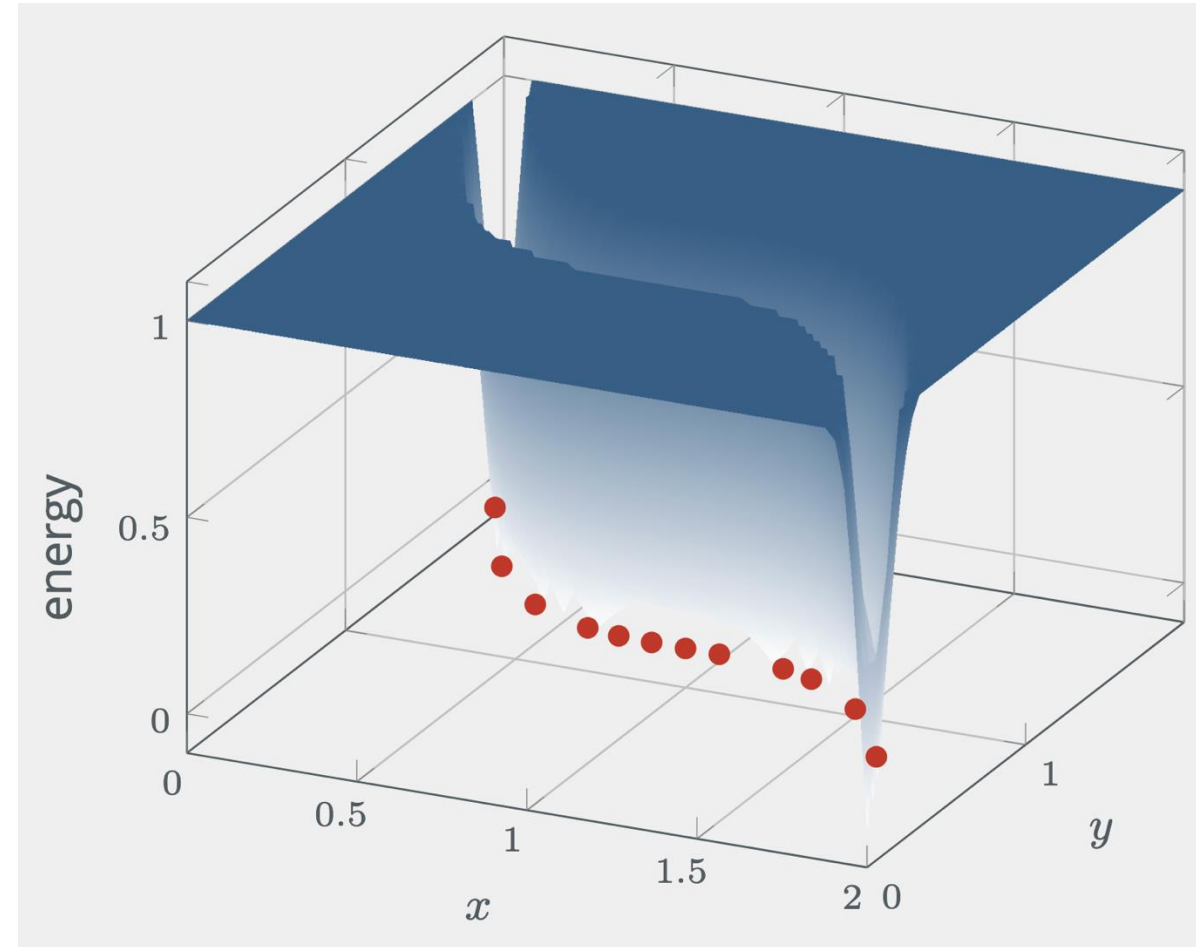
This generic definition fits a large majority of machine learning models. For example $\mathcal{L}(E(\mathbf{x}), \mathbf{y})$ (a classifier)



Definition: energy-based models

GANs are also energy models. The generator G generates samples off the manifold, then the discriminator D says these should be one everywhere, whereas it says real samples should be zero everywhere.

The generator also has to get good at sampling points on the data manifold. So it has to learn to generate points in the valley regions.



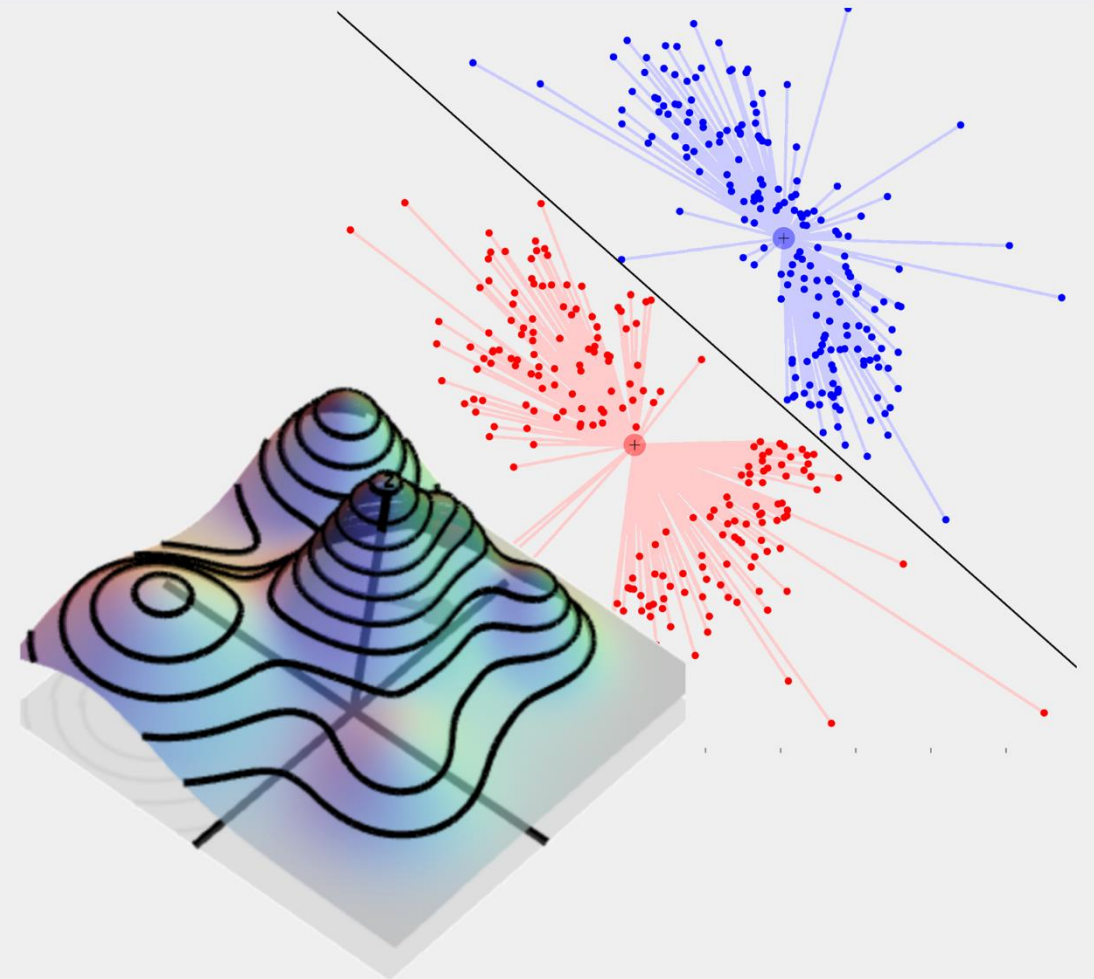
Definition: clustering algorithm

A cluster is a **connected-component** of a **level-set** of the **unknown PDF** over our data observations.

Traditionally:

- We don't know the PDF (the energy landscape)
- We don't necessarily know the level set
 - although 0.5 is appropriate for BCE
- This can be expensive (deep learning)

Example: clustering by its definition





Definition: softmax and softmin

Softmax and softmin functions rescale elements to be in the range $[0, 1]$ and such that they sum to 1. So they create a probability mass function, e.g.:

$$\begin{bmatrix} 1.3 \\ 7.2 \\ 2.4 \\ 0.5 \\ 1.1 \end{bmatrix} \rightarrow \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \rightarrow \begin{bmatrix} 0.0027 \\ 0.9858 \\ 0.0081 \\ 0.0012 \\ 0.0022 \end{bmatrix}$$

Softmax functions are widely used (not just for EBMs) where a distribution is needed, such as the last layer of a classifier.

EBMs are based on the observation that any probability density function $p(\mathbf{x})$ for $\mathbf{x} \in \mathbb{R}^n$ can be expressed as:

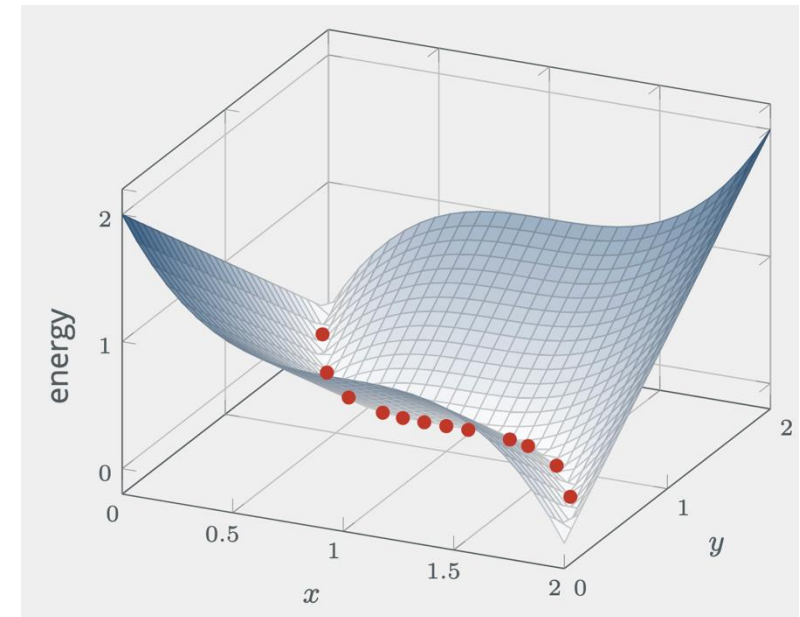
$$p(\mathbf{x}) = \frac{e^{-E(\mathbf{x})}}{\int_{\tilde{\mathbf{x}} \in \mathcal{X}} e^{-E(\tilde{\mathbf{x}})} d\tilde{\mathbf{x}}}$$

where $E(\mathbf{x}) : \mathbb{R}^n \rightarrow \mathbb{R}$ is the energy function.

computation of the integral is intractable!

Approximation:

- Boltzmann machines and variants!
- Sample our model in a way that goes off the manifold – e.g. “diffuse” away from the manifold.





Definition:

Boltzmann machines are one of the earliest neural networks for modelling binary data. They can associate the probability of the visible vectors $\mathbf{v}$ using finite summations:

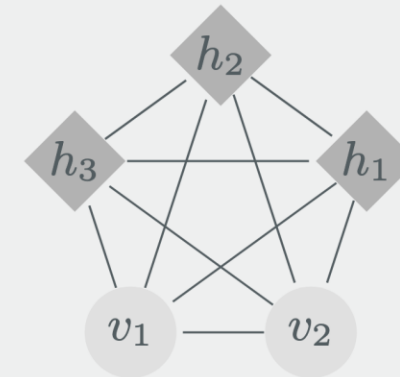
$$p_{\theta}(\mathbf{v}) = \frac{\sum_{\mathbf{h}} e^{-\beta E_{\theta}(\mathbf{v}, \mathbf{h})}}{\sum_{\tilde{\mathbf{v}}} \sum_{\mathbf{h}} e^{-\beta E_{\theta}(\tilde{\mathbf{v}}, \mathbf{h})}}$$

They are typically trained via negative log-likelihood through contrastive divergence, where the weights are updated:

$$\sum_{\mathbf{x} \in \mathcal{X}} \frac{\partial \ln p(\mathbf{x})}{\partial w_{i,j}} = \mathbb{E}_{p_d}[\mathbf{v}\mathbf{h}^T] - \mathbb{E}_{p_{\text{model}}}[\mathbf{v}\mathbf{h}^T]$$

Example: Boltzmann machine

They are an energy model which just have visible layers $v_1, v_2, \dots, v_n$ (inputs) and hidden layers $h_1, h_2, \dots, h_n$ (no outputs):



This example Boltzmann Machine has 2 visible units and 3 hidden units.



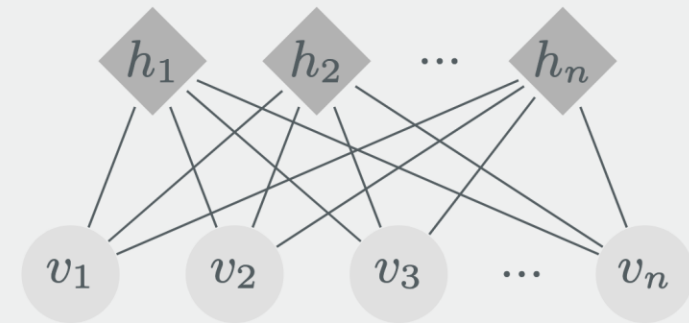
Definition: RBMs and DBMs

Restricted Boltzmann Machines (RBMs) and Deep Boltzmann Machines (DBMs) are Boltzmann machines with a more restricted (bipartite) graph structure. DBMs have additional hidden layers.

That means that the visible units conditional on the hidden units become independent, which makes training these straightforward in practice.

Example: RBM

RBMs have a restricted architecture so that there are no connections between hidden units:



DBMs are like the above, but with multiple hidden layers between.



Restricted Boltzmann Machine

Code on GitHub:

https://github.com/atapour/dl-pytorch/blob/main/Restricted_Boltzmann_Machine/Restricted_Boltzmann_Machine.py.ipynb

Code on Colab:

https://colab.research.google.com/github/atapour/dl-pytorch/blob/main/Restricted_Boltzmann_Machine/Restricted_Boltzmann_Machine.py.ipynb

Contrastive Divergence



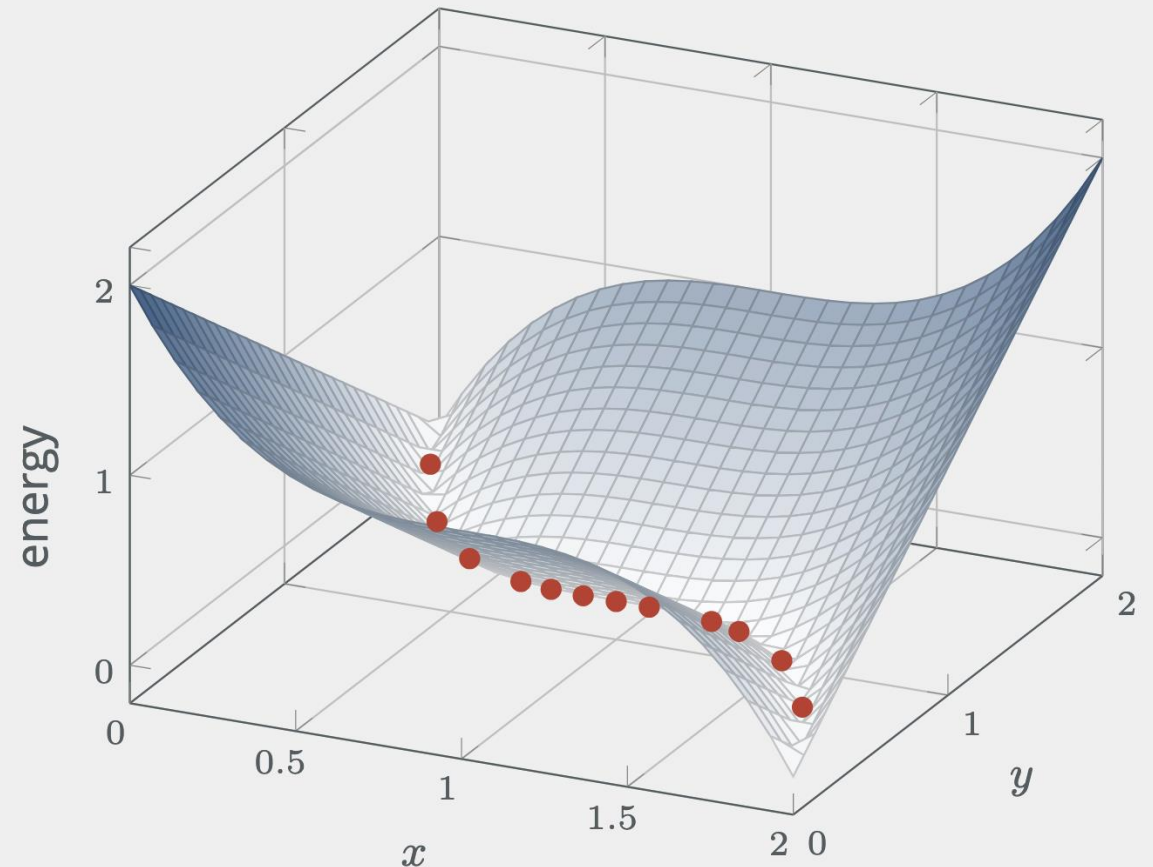
Definition: contrastive-divergence

The gradient of the negative log-likelihood loss $\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x} \sim p_d}[-\ln p_\theta(\mathbf{x})]$ has been shown to demonstrate the following property:

$$\nabla_\theta \mathcal{L} = \mathbb{E}_{\mathbf{x}^+ \sim p_d}[\nabla_\theta E_\theta(\mathbf{x}^+)] - \mathbb{E}_{\mathbf{x}^- \sim p_\theta}[\nabla_\theta E_\theta(\mathbf{x}^-)]$$

where $\mathbf{x}^- \sim p_\theta$ is a sample from the energy model found through a Monte Carlo Markov Chain (MCMC) generating procedure.

Energy increases off manifold



Denoising Diffusion Models



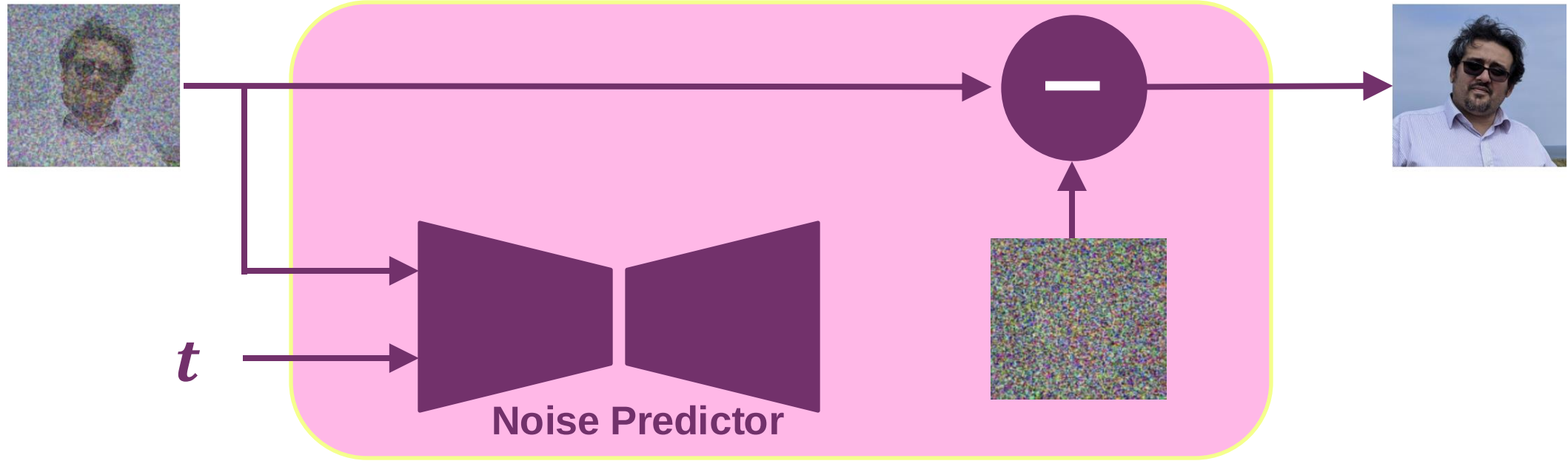
consist of two processes:

- Forward diffusion process that gradually adds noise to input
- Reverse denoising process that learns to generate data by denoising



(generative)

Denoising Diffusion Probabilistic Model

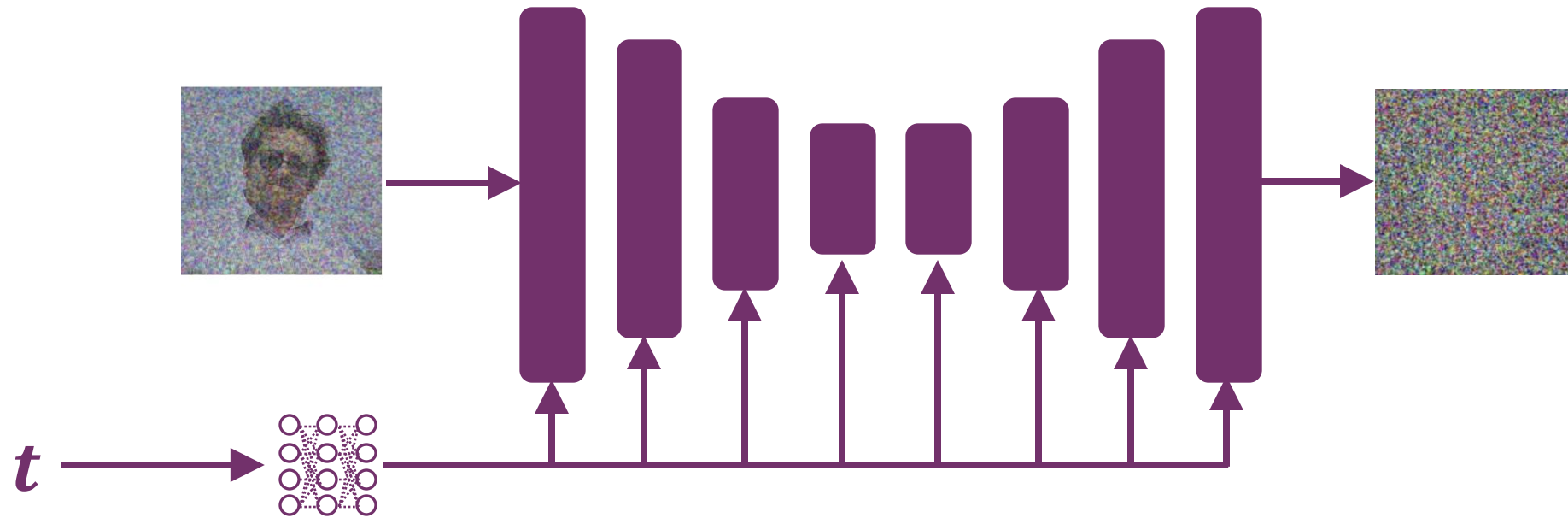


Algorithm 1 Training

- 1: **repeat**
- 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
- 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
- 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 5: Take gradient descent step on
$$\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$$
- 6: **until** converged

Algorithm 2 Sampling

- 1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 2: **for** $t = T, \dots, 1$ **do**
- 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
- 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
- 5: **end for**
- 6: **return** $\mathbf{x}_0$



Time

- sinusoidal positional embeddings or random Fourier features.
 - fed to the residual blocks via addition or using adaptive group normalisation!
- U-Net architecture
 - Residual blocks
 - Various schedules are possible!

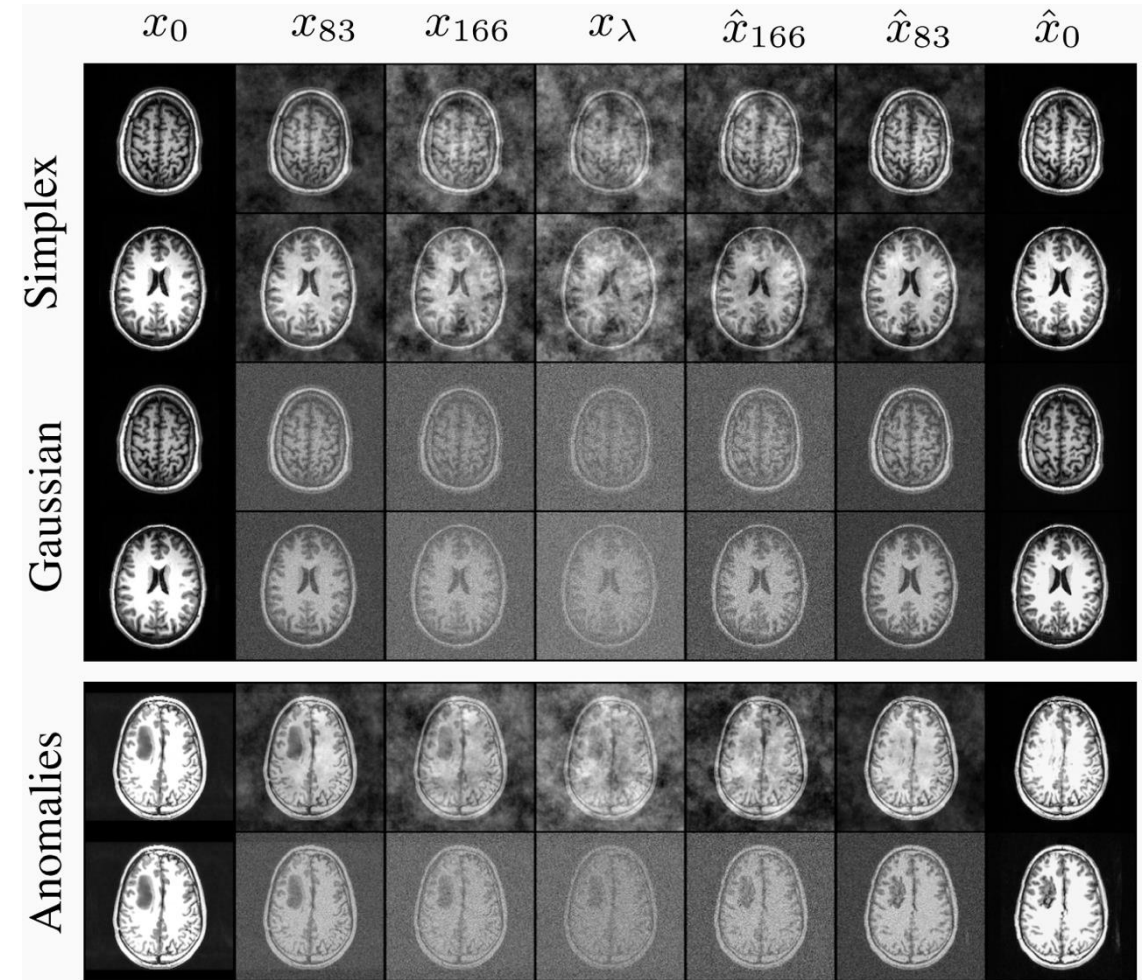
Diffusion-based Anomaly Detection



Diffusion-based anomaly detection

Like GANs, diffusion-based models work well for anomalies (great for small datasets).

- Do a partial diffusion
- Train only on healthy/normal data
- Abnormal denoising will only know how to make the data look normal
- Any error = surprise = anomalies



What we learned today!



Summary

- The Trilemma
- Energy-based Model
- Boltzmann Machines
- Diffusion Models
- Applications